Math behind `TenShadows`

Stochastic Batman

August 28

Abstract

`TenShadows` routes pedestrians through shadows rather than along the shortest path. It does this by attaching to every street segment a number between 0 and 1 describing how much of that segment currently lies in the shadow of a building, and then letting the user trade distance against sun exposure. This document derives every piece of this program from scratch. The grammar and some parts of the structure in this document have been corrected by Claude Sonnet 5, though the math and intuition are all mine.

1 What this document covers

This document assumes you are comfortable with graph algorithms and uncomfortable with geospatial mathematics, which is the opposite of the usual assumption. Accordingly, *graph traversal is not covered here*: Dijkstra's algorithm, A*, priority queues, and the machinery of shortest-path search are taken as known, and nothing in this document will explain them. What I do cover is everything that happens *before* the search runs and everything that determines *what the search is optimising*, namely: the coordinate terminology a newcomer to the domain needs (Section 2), where the sun is and how I describe that (Section 3), the derivation of a building's shadow (Sections 4–5), how a shadow becomes a number attached to a street segment (Section 6), and the edge cost function together with a proof that it is safe to hand to A* (Section 7). Every symbol introduced is collected in the notation table of Section 9. The one place the two worlds touch is Section 7, where I show the cost function is non-negative and that straight-line distance remains an admissible heuristic.

2 Terminology

2.1 Latitude and longitude

The Earth is very close to an ellipsoid, and a point on its surface is named by two angles. *Latitude*, written φ, is the angle north or south of the equator, ranging from -90° at the south pole to $+90^\circ$ at the north pole. *Longitude*, written θ, is the angle east or west of the prime meridian through Greenwich, ranging over $(-180^\circ, 180^\circ]$. For example, Tbilisi sits near $\varphi \approx 41.7^\circ$, $\theta \approx 44.8^\circ$, Saarbrücken near $\varphi \approx 49.2^\circ$, $\theta = 7.0^\circ$ (yes, up to 4 decimal places, Saarbrücken's longitude is 7.0°).

One degree of longitude is not one degree of latitude. A degree of latitude is essentially constant at about $111,320$m, because meridians are great circles (though the exact distance varies from about $110,574$ meters at the equator to $111,694$ meters at the poles). A degree of longitude, however, is measured along a circle of latitude whose radius shrinks as you move away from the equator by a factor of $\cos\varphi$, so

$$\text{meters per degree of longitude} \approx 111{,}320 \cdot \cos\varphi,$$

which is roughly 83,100m in Tbilisi and 72,700m in Saarbrücken. Any calculation that treats raw longitude and latitude values as though they were Cartesian coordinates in meters is therefore wrong, and wrong by a different amount in every city.

2.2 Coordinate reference systems and EPSG codes

A *coordinate reference system* (CRS) is the specification that tells you what a pair of numbers means: which ellipsoid models the Earth, where the origin sits, what the units are, and how the curved surface was flattened onto a plane, if it was. Two identical-looking pairs $(44.8, 41.7)$ interpreted under different CRSs denote different places, so a bare coordinate pair without a CRS is as meaningless as a byte string without an encoding.

The EPSG registry (originally the European Petroleum Survey Group) assigns a small integer to each such system, and that integer is the universal way to name one.

Two families matter for `TenShadows`. `EPSG:4326`, known as WGS 84, is the geographic system in which coordinates are longitude and latitude in degrees on the reference ellipsoid. This is what GPS reports, what OpenStreetMap stores, and what GeoJSON requires. **UTM zones** are projected systems in which coordinates are eastings and northings in meters on a flat plane, obtained by slicing the globe into 6°-wide strips and projecting each strip separately so that distortion within a strip stays small. Tbilisi falls in UTM zone 38N (`EPSG:32638`) and Saarbrücken in zone 32N (`EPSG:32632`).

The working rule for the entire pipeline follows immediately: **do all geometry in a projected metric CRS, and store or export only in `EPSG:4326`**. Shadow lengths, translations, intersections, and distances are all metric operations and must happen in UTM. Python's `GeoPandas` library can pick the right zone automatically via `gdf.estimate_utm_crs()`.

2.3 Altitude and azimuth

Latitude and longitude locate *you on the Earth.* To describe where the sun is, an entirely different coordinate system is needed that locates something (cough... cough... the sun...) in the sky, with its origin at wherever you happen to be standing. This is the *horizontal* or *topocentric* system, and it also uses two angles.

The *altitude* a is the angle of the object above the horizon: $a = 0°$ means on the horizon, $a = 90°$ means directly overhead, and $a < 0°$ means below the horizon, which for the sun means night. The *azimuth* A is the compass direction you would turn to face, measured clockwise from north, so $A = 0°$ is north, 90° east, 180° south, and 270° west. Together the pair (a, A) pins a single point in the sky, just as (φ, θ) pins a single point on the globe.

Remark 1. The word *altitude* is overloaded and the collision is worth flagging. In aviation and in most GIS contexts, altitude means height above sea level, measured in meters. In astronomy, and throughout this document, altitude means an *angle* above the horizon. `TenShadows` uses the astronomical sense exclusively. Building heights are always written h and always in meters.

The division of labor between the two angles is what makes shadows computable: **altitude alone determines how long a shadow is, and azimuth alone determines which way it points**. A low sun makes long shadows regardless of compass direction; a southern sun throws shadows north regardless of how high it sits. The two effects are independent, as you will see in Sections 4 and 5.

3 Where is the sun?

3.1 The solar position oracle

Computing (a, A) from a timestamp and a location is a solved problem in astronomy involving the Earth's orbital elements, its axial tilt, and the equation of time. `TenShadows` does not

re-derive it and this document will not either; I treat it as an oracle

$$\text{sun} : (\text{timestamp}, \varphi, \theta) \longmapsto (a, A),$$

implemented by the SunCalc algorithm, which is accurate to well within a tenth of a degree over the range of dates anyone will plausibly query. That accuracy is irrelevant next to the error in our building heights, so there is nothing to gain from a more elaborate model.

Remark 2. `TenShadows` deliberately uses `suncalc-py` in the Python pipeline and `suncalc` in the browser, rather than a different library on each side. The reason is not accuracy but *convention*. SunCalc reports azimuth in radians measured clockwise *from south*, whereas the other Python astronomy library I have heard of (`pvlib`) reports degrees measured clockwise *from north*. Mixing the two conventions rotates every shadow in the city by exactly 180°, which is obviously not something we want. Where SunCalc's south-based angle A_{sc} must be converted to the north-based A used throughout this document, the conversion is

$$A = (A_{\text{sc}} + \pi) \mod 2\pi.$$

Every formula below assumes the north-based convention.

3.2 The sun direction vector

Fix a *local East-North-Up* (ENU) frame: a right-handed Cartesian frame with its origin at our location, $\hat{x}$ pointing east, $\hat{y}$ pointing north, and $\hat{z}$ pointing straight up, with all three measured in meters. Since a UTM projection acts almost like a rigid metric map over the short distances we encounter on a walking route (meaning distances and angles are preserved locally), I can safely treat the ENU horizontal plane and the projected UTM plane as one and the same - both just $\mathbb{R}^2$.

Let $\hat{s} \in \mathbb{R}^3$ be the unit vector pointing from the observer *towards* the sun. It rises at angle a above the horizontal plane, so it splits into a vertical component $\sin a$ and a horizontal part of length $\cos a$. That horizontal part points along the bearing A, whose east and north components are $\sin A$ and $\cos A$. Putting the three together,

$$\hat{s} = \big(\cos a \sin A,\ \cos a \cos A,\ \sin a\big).$$

As a check, a sun on the eastern horizon has $a = 0$, $A = 90°$ and gives $\hat{s} = (1, 0, 0)$, which points due east; a sun due south has $A = 180°$ and gives a horizontal part $(0, -\cos a)$, which points due north-negative, that is, south. Both are correct.

We will also need the normalized horizontal projection of $\hat{s}$, which is the unit vector in the horizontal plane pointing towards the sun:

$$\hat{u} = \frac{(\cos a \sin A,\ \cos a \cos A)}{\cos a} = (\sin A,\ \cos A), \qquad a \neq 90°.$$

Note that $\hat{u}$ depends only on the azimuth, exactly as the informal claim at the end of Section 2 promised.

4 How long is a shadow?

4.1 The intuition

Stand a vertical pole of height h on flat ground with the sun at altitude a. The pole, its shadow, and the sun ray grazing the pole's tip form a right triangle: the pole is the vertical leg of length h, the shadow is the horizontal leg of unknown length L, and the ray is the hypotenuse. The angle between the ray and the ground is precisely the solar altitude a (that is what altitude means).

In that triangle h is opposite a and L is adjacent to it, so $\tan a = h/L$, giving $L = h/\tan a$. Everything else in this section is that observation done carefully in three dimensions so that it also handles direction, non-vertical geometry, and the degenerate cases.

The formula behaves the way experience says it should. At noon in summer a is large, $\tan a$ is large, and shadows are short stubs at the feet of buildings. Near sunrise and sunset $a \to 0^+$, $\tan a \to 0^+$, and $L \to \infty$: shadows stretch across the whole city, which is both physically real and numerically dangerous. When $a \geq 90^\circ$ is approached the shadow collapses to nothing, and when $a \leq 0$ the sun has set and the notion of a building shadow stops meaning anything at all.

4.2 The derivation

Strictly speaking sunlight is not perfectly parallel: the sun is a disc in the sky, not a point, so rays reaching different spots fan out ever so slightly. But the sun is far away and looks tiny, only about half a degree across, so this fanning is negligible: two points 200m apart receive light from directions differing by under 10^{-6} radians, which shifts a shadow's edge by well under a millimetre. That is nothing compared to the uncertainty of several meters in our building heights, so I simplify and treat every ray as travelling in the same direction. If $\hat{s}$ is the direction pointing towards the sun, light itself travels the opposite way, so every ray's direction is $-\hat{s}$.

Proposition 1 (Shadow length)**.** *Let a point sit at height $h > 0$ above horizontal position $p \in \mathbb{R}^2$ on flat ground at elevation zero, and let the sun be at altitude $a \in (0^\circ, 90^\circ)$ and azimuth A. Then the shadow of that point falls at*

$$p + d, \text{ where } d = -\frac{h}{\tan a}\,(\sin A,\, \cos A) \in \mathbb{R}^2,$$

so the shadow lies at horizontal distance $L = \dfrac{h}{\tan a}$ from p, in the direction exactly opposite the sun.

Proof. Write the elevated point as $P = (p_x, p_y, h) \in \mathbb{R}^3$ and follow the light ray leaving it. The ray travels in direction $-\hat{s}$, so its points are

$$q(t) = (p_x,\, p_y,\, h) + t\,\big(-\cos a \sin A,\, -\cos a \cos A,\, -\sin a\big), \qquad t \geq 0.$$

The ground is the plane $q_z = 0$, so $h - t \sin a = 0$ has the unique positive root (since $a \in (0^\circ, 90^\circ)$):

$$t^\star = \frac{h}{\sin a}$$

Substituting $t^\star$ into the horizontal components gives the displacement from p to the shadow point,

$$d = -\frac{h}{\sin a}\,\big(\cos a \sin A,\, \cos a \cos A\big) = -\frac{h \cos a}{\sin a}\,(\sin A, \cos A) = -\frac{h}{\tan a}\,(\sin A, \cos A),$$

Its magnitude is $\|d\| = \frac{h}{\tan a}\|(\sin A, \cos A)\| = \frac{h}{\tan a}$, since $\sin^2 A + \cos^2 A = 1$, and its direction is $-\hat{u}$, which is antiparallel to the horizontal direction of the sun. □

The minus sign carries the physical content. Take the sun due south, $A = 180^\circ$, which is where it sits at midday anywhere in the northern hemisphere. Then $(\sin A, \cos A) = (0, -1)$ and

$$d = -L\,(0, -1) = (0,\, +L),$$

which is a displacement of $+L$ in the northern component: the shadow points due north. That is correct, and it is a cheap unit test that catches both a flipped sign and a swapped azimuth convention.

4.3 Clamping the degenerate cases

Proposition 1 excluded $a \le 0$ and $a = 90°$, and near the excluded lower endpoint the formula is valid but useless. As $a \to 0^+$ we get $L \to \infty$, which in floating point yields shadow polygons that are kilometers long, dominated entirely by the error in a and in h. `TenShadows` therefore applies two clamps. When $a \le 0$ the sun is below the horizon and I set $\sigma(e) = 1$ (see Definition 1 in Section 6) for every edge by convention, since at night the shade term carries no information and the cost function of Section 7 degenerates gracefully to pure distance. When $0 < a < a_{\min}$ with $a_{\min} = 5°$, I cap the shadow length at $L_{\max} = 200$ m, that is, I use

$$L = \min\left(\frac{h}{\tan a},\, L_{\max}\right).$$

Both constants are pragmatic rather than principled: at such low sun angles the flat-ground assumption of Section 8 has already failed badly, and no choice of cap makes the output trustworthy.

5 From a building to a shadow polygon

5.1 The building model

Proposition 1 casts a single point. A building is a solid, and I need the shadow of the whole thing. `TenShadows` uses the standard *LoD1* model, which is what OpenStreetMap data supports: building k is an extruded prism with polygonal footprint $B_k \subset \mathbb{R}^2$ in projected meters, vertical walls, a flat roof, and a single height h_k in meters, occupying the solid region

$$V_k = \{(p, z) : p \in B_k,\, 0 \le z \le h_k\} \subset \mathbb{R}^3.$$

Pitched roofs, setbacks, overhangs, and towers are all flattened into one number. Since h_k itself typically comes from an OSM `building:levels` tag multiplied by an assumed storey height, the model's crudeness is dominated by the input's crudeness anyway.

5.2 The shadow is a swept region, not a translated copy

Here is the mistake that looks correct and is not. Having computed the displacement d_k for building k from its height h_k via Proposition 1, one is tempted to write the shadow as the translated footprint $B_k + d_k$. But $B_k + d_k$ is the shadow of the *roof outline* only, a copy of the footprint sitting off at the far end of the shadow, with the entire region between the building and that copy left incorrectly sunlit.

The fix comes from asking what casts the missing part. A point on a vertical wall at height $z \in [0, h_k]$ sits above some footprint point $p \in \partial B_k$ (boundary of B_k), and by Proposition 1 applied with height z its shadow lands at $p + \dfrac{z}{h_k} d_k$, because the displacement is linear in the height. As z sweeps from 0 up to h_k the scale factor $\frac{z}{h_k}$ sweeps from 0 to 1, so the wall paints a continuum of translated copies of the footprint rather than just the last one. The shadow of the whole solid is therefore the union over all of them:

$$S_k = \bigcup_{t \in [0,1]} (B_k + t\, d_k).$$

Recall that the *Minkowski sum* of two sets is $X \oplus Y = \{x + y : x \in X,\, y \in Y\}$. Writing $[0, d_k] = \{t\, d_k : t \in [0, 1]\}$ for the straight segment from the origin to d_k, the union above is precisely

$$S_k = B_k \oplus [0, d_k],$$

the footprint dragged along the shadow displacement. This is the correct shadow of building k on flat ground.

5.3 Computing the Minkowski sum

For a polygon with vertices $v_1, \ldots, v_n$ traversed around the boundary ∂B_k, the sum $B_k \oplus [0, d_k]$ can be assembled exactly as a union of $n+2$ simple pieces: the original polygon B_k, its translate $B_k + d_k$, and for each boundary edge (v_i, v_{i+1}) the parallelogram

$$Q_i = \mathrm{hull}\big(v_i,\, v_{i+1},\, v_{i+1} + d_k,\, v_i + d_k\big)$$

swept by that edge as it slides along d_k. Taking the union of these with Shapely's `union_all` gives S_k directly, and each piece is convex and trivially constructed, so nothing subtle is required.

Remark 3. A widespread shortcut computes $\mathrm{hull}(B_k \cup (B_k + d_k))$, the convex hull of the footprint together with its translate. This is correct if and only if B_k is convex. For any building with a re-entrant corner, the hull fills in the concavity and reports shade in places that are in fact sunlit, which biases routes towards exactly the dense old-town geometry where the error is largest.

6 From shadows to a number on an edge

6.1 The shadow fraction

Let the city contain buildings $k = 1, \ldots, K$ with shadows $S_1, \ldots, S_K$ computed as above for the current sun position. The lit-or-shaded status of a point depends on whether *any* building shades it, so the object of interest is the union

$$S = \bigcup_{k=1}^{K} S_k \subset \mathbb{R}^2.$$

Taking the union before measuring anything is not optional: adjacent buildings on a narrow street cast heavily overlapping shadows, and summing their contributions separately would count the same shaded pavement several times and can easily produce fractions above 1.

Now let e be a street segment (an edge), represented as a polyline in the projected CRS, and write $\ell(X)$ for the total length of a set $X \subseteq \mathbb{R}^2$ that is a finite union of curve pieces, so that $\ell(e)$ is the segment's length in meters and $\ell(e \cap S)$ is the total length of the possibly-disconnected portion of it that lies in shade.

Definition 1 (Shadow fraction)**.** The shadow fraction of edge e under sun position (a, A) is

$$\sigma(e) = \frac{\ell(e \cap S)}{\ell(e)} \in [0, 1],$$

with $\sigma(e) = 1$ by convention whenever $a \le 0$.

The two extremes are worth naming: $\sigma(e) = 0$ means the segment is in full sun over its entire length, and $\sigma(e) = 1$ means it is fully shaded. Because $e \cap S \subseteq e$, we always have $\ell(e \cap S) \le \ell(e)$, so σ genuinely lands in $[0, 1]$ and needs no further clamping.

6.2 Sampling, and why exactness would be wasted

Computing $\ell(e \cap S)$ exactly requires a polygon-polyline overlay against a union of tens of thousands of polygons, which is expensive in Python and considerably worse in JavaScript. `TenShadows` instead samples. Place $m = \left\lceil \dfrac{\ell(e)}{\Delta} \right\rceil$ points $p_1, \ldots, p_m$ at spacing Δ along e and estimate

$$\hat{\sigma}(e) = \frac{1}{m} \sum_{i=1}^{m} \mathbf{I}_{\{p_i \in S\}},$$

where $\mathbf{I}_{\{\cdot\}}$ is the indicator, equal to 1 when its argument holds and 0 otherwise. The estimator misclassifies only the sampling intervals straddling a shadow boundary, so its error is bounded by $\Delta \cdot (\text{number of boundary crossings along } e)/\ell(e)$, which for $\Delta = 5\text{m}$ and a typical block with a handful of crossings sits well under a percent. Since building heights are themselves uncertain by several meters, refining Δ below this buys precision the inputs cannot support.

6.3 The dual form: ray casting

The sampling estimator still asks the membership question $p \in S$, which appears to need the union S built explicitly. It does not, and the identity that removes it is the practical heart of the browser implementation.

Proposition 2 (Ray-casting criterion)**.** *Let $a \in (0°, 90°)$, let $\hat{u} = (\sin A, \cos A)$ be the horizontal direction towards the sun, and for each building k (of height h_k) let $L_k = \frac{h_k}{\tan a}$. Then a point $p \in \mathbb{R}^2$ satisfies $p \in S$ if and only if there exists a building k and a distance $\tau \in [0, L_k]$ such that*

$$p + \tau\, \hat{u} \in B_k.$$

Proof. By definition $p \in S$ iff $p \in S_k$ for some k, and $p \in S_k = \bigcup_{t \in [0,1]} (B_k + t d_k)$ iff there is a $t \in [0, 1]$ with $p - t\, d_k \in B_k$. By Proposition 1, $d_k = -L_k \hat{u}$, so $p - t\, d_k = p + t L_k \hat{u}$. Substituting $\tau = t L_k$, which ranges over $[0, L_k]$ as t ranges over $[0, 1]$, gives the stated condition. □

The reading is exactly the physical one: to decide whether you are standing in shade, look towards the sun and walk in that direction; you are shaded precisely if you run into a building before going further than that building's own shadow length. Equivalently, and more suggestively, the sun ray reaching p climbs at slope $\tan a$, so at horizontal distance τ it is at height $\tau \tan a$, and building k blocks it exactly when the ray passes over the footprint while still below the roof, that is, when $\tau \tan a \le h_k$.

Computationally this replaces a global polygon union with an independent, short, bounded ray query per sample point, answerable against a static R-tree over the footprints. That is cheap enough to run in a Web Worker whenever the user moves the time slider, which is what lets `TenShadows` ship building geometry to the client and compute shade live rather than pre-baking a fraction that would be valid for only one instant.

7 The edge cost function

7.1 Definition and interpretation

Let $w \ge 0$ be the user's *shade preference*, set by a slider in the interface. The cost assigned to edge e is

$$c(e) = \ell(e)\,\big(1 + w\,(1 - \sigma(e))\big).$$

The factor $1 - \sigma(e)$ is the *sunlit* fraction, so the multiplier $1 + w(1 - \sigma(e))$ ranges from 1 on a fully shaded segment up to $1 + w$ on a fully exposed one. Setting $w = 0$ makes the multiplier identically 1 and recovers the ordinary shortest path, so the shade-aware router contains the plain router as a special case.

The parameter w has a simple, practical meaning: it tells the router how much extra walking is worth one meter of sun avoided. A fully sunlit segment of length ℓ is treated as costing the same as $(1 + w)\ell$ meters of fully shaded walking. That means the router is willing to take a shaded detour up to $(1 + w)$ times as long as the sunlit route it replaces. So if $w = 1$, you are willing to walk twice as far to stay fully in shade. If $w = 0.2$, you are willing to accept a twenty percent longer route.

It is often useful to display alongside the route its total sun exposure, which in the same notation is $\sum_{e \in P} \ell(e)(1 - \sigma(e))$ meters for a path P, and which is a physically meaningful quantity independent of w.

7.2 Why this form and not the other one

The obvious alternative is to *discount* shaded edges rather than penalise sunlit ones, writing $c'(e) = \ell(e)(1 - w\,\sigma(e))$ with $0 \le w < 1$. It has the same monotonicity, it is still positive, and it ranks edges identically for fixed length. It is nevertheless the wrong choice, and the reason is Proposition 3 below: because its multiplier is strictly less than 1, a path's cost can fall below its true geometric length, and straight-line distance stops being a lower bound on the remaining cost. A* run with a Euclidean heuristic against c' can then terminate on a suboptimal path, and the bug is invisible in testing because the returned routes still look sensible. The form actually used has a multiplier bounded below by 1, which is precisely what rescues the heuristic.

Proposition 3 (Non-negativity and A* admissibility)**.** *Let $x_v \in \mathbb{R}^2$ denote the projected position of node v, let z be the target node with position x_z, and define the heuristic $h(v) = \|x_v - x_z\|$. Under the cost c above with $w \ge 0$: (i) $c(e) \ge \ell(e) \ge 0$ for every edge, so Dijkstra's algorithm applies; (ii) h is admissible; and (iii) h is consistent.*

Proof. For (i), recall $c(e) = \ell(e)\,\big(1 + w(1 - \sigma(e))\big)$, where I call the factor $1 + w(1 - \sigma(e))$ the multiplier. Since $\sigma(e) \in [0, 1]$ gives $1 - \sigma(e) \ge 0$, and $w \ge 0$, we get $w(1 - \sigma(e)) \ge 0$, so the multiplier is at least 1; multiplying by $\ell(e) \ge 0$ preserves the inequality. For (ii), let P be any path from v to the target z. Then

$$\sum_{e \in P} c(e) \;\ge\; \sum_{e \in P} \ell(e) \;\ge\; \|x_v - x_z\| = h(v),$$

where the first inequality is (i) and the second holds because the total length of a polyline from x_v to x_z is at least the straight-line distance between its endpoints, by the triangle inequality. This holds for every path P from v to z, so it holds in particular for the cheapest one. Write $h^\star(v) := \min_P \sum_{e \in P} c(e)$ for the true cost of that cheapest path, the quantity A* is actually trying to find. Taking the minimum over all P on the right then gives $h(v) \le h^\star(v)$: the heuristic never overestimates the true remaining cost, which is precisely admissibility. For (iii), let $e = (v, v')$ be an edge; then

$$h(v) - h(v') = \|x_v - x_z\| - \|x_{v'} - x_z\| \;\le\; \|x_v - x_{v'}\| \;\le\; \ell(e) \;\le\; c(e),$$

the first step by the reverse triangle inequality and the second because an edge's polyline is at least as long as the straight line between its endpoints. This is the consistency condition $h(v) \le c(e) + h(v')$. □

Note finally that σ enters the cost only through a per-edge multiplication, and that σ itself depends on the sun but not on w. Recomputing shade after a time change is therefore expensive but rare, while re-weighting after a slider change is a single multiply per edge and can be done on every frame.

8 Assumptions and limitations

The ground is assumed *flat and at elevation zero*, which is what lets Proposition 1 solve $q_z = 0$; on Tbilisi's hills this is materially wrong, since a shadow cast downhill runs long and one cast uphill is cut short, and correcting it requires a digital elevation model and ray-marching against terrain rather than a single plane intersection. Buildings are assumed to be *opaque flat-roofed prisms* of a single height, which discards pitched roofs and arcades and is anyway finer than the height data deserves. Buildings are assumed to be the *only* occluders, which omits trees, and on residential streets trees are very often the dominant source of shade. Unfortunately, OpenStreetMap doesn't carry `natural=tree` tags consistently. As the source of light, only *direct* sunlight is modeled. Finally, and most consequentially, *building heights come from OSM*

tags that are sparse in many cities. Ironically, neither Tbilisi nor Saarbrücken has enough data to escape this problem: querying OpenStreetMap directly (as of August 28, 2026, a figure that will drift as the map is edited) shows that only 0.23% of Tbilisi's buildings and 0.14% of Saarbrücken's carry an explicit `height` tag, while `building:levels` is present on roughly 15% and 19% of them respectively, leaving the large majority of buildings in both cities to the type-based default. Where `height` is absent `TenShadows` falls back to `building:levels` multiplied by an assumed storey height and then to that default.

9 Notation

Symbol	Meaning	Units
φ	Latitude, north positive	degrees
θ	Longitude, east positive	degrees
a	Solar altitude, angle above the horizon	degrees
A	Solar azimuth, clockwise from north	degrees
A_{sc}	Solar azimuth as SunCalc reports it, clockwise from south	radians
δ	Solar declination	degrees
$\hat{s}$	Unit vector towards the sun, in the ENU frame	—
$\hat{u}$	Horizontal unit vector towards the sun, $(\sin A, \cos A)$	—
h, h_k	Building height	meters
L, L_k	Shadow length, $h/\tan a$	meters
$L_{\max}$	Cap on shadow length at low sun	meters
$a_{\min}$	Altitude below which the cap applies	degrees
d, d_k	Shadow displacement vector, $-L\,\hat{u}$	meters
B_k	Footprint polygon of building k, projected	—
V_k	Solid prism occupied by building k	—
S_k	Shadow of building k, $B_k \oplus [0, d_k]$	—
S	Union of all building shadows	—
$X \oplus Y$	Minkowski sum, $\{x + y : x \in X, y \in Y\}$	—
e	A street segment, as a projected polyline	—
$\ell(\cdot)$	Total length of a curve or union of curves	meters
$\sigma(e)$	Shadow fraction of edge e	—
$\hat{\sigma}(e)$	Sampled estimate of $\sigma(e)$	—
Δ	Sampling spacing along an edge	meters
w	Shade preference set by the user	—
$c(e)$	Cost of edge e	meters
x_v	Projected position of node v	meters
$h(v)$	A* heuristic, straight-line distance to the target	meters
$h^\star(v)$	True cost of the cheapest path from v to the target under c	meters